

Documentação do Jogo "Echoes of the Forest"

Professor

- Vinícius Humberto Serapilha Durelli

Alunos

- Carlos Gabriel Teixeira
- Alvaro Cordeiro Ribeiro
- Rafael Castro Moreira

Nesta documentação resumimos as nossas classes, módulos e funções, para melhores explicações da parte mais complexa recomendamos que leia os comentários dentro do código do jogo.

Estrutura do Projeto

Nosso jogo é dividido em pastas principais:

- `src/` - Contém o código-fonte do jogo.
- `content/` - Armazena imagens, fontes, sons e o mapa do jogo.

Dentro da pasta `src/`, encontramos mais subpastas e o arquivo `play.py`. As subpastas incluem:

- `src/components/` - Contém os componentes principais do jogo, como jogador, inimigos e interface.
 - `src/core/` - Contém os sistemas centrais, responsáveis por gerenciar aspectos fundamentais do jogo, como física, câmera, entrada de usuário e lógica geral.
 - `src/data/` - Armazena dados e configurações do jogo.
 - `src/estagios/` - Contém os estágios e a lógica de progresso do jogo.
-

1. Módulos e Classes

`src/components/`

`enemy.py`

- **Classe Enemy:** Responsável por controlar o comportamento dos fantasmas inimigos.

- `update()` - Atualiza a posição e animação do inimigo.
- `move()` - Determina a direção do inimigo.

player.py

- **Classe Player:** Controla o personagem do jogador.
 - `update()` - Atualiza a posição e animação do jogador.
 - `handle_input()` - Verifica quais teclas estão pressionadas.
 - `load_spritesheet()` - Carrega as animações do jogador.

label.py

- **Classe Label:** Controla os textos exibidos na tela.
 - `render()` - Renderiza o texto na tela.

menu.py

- **Classe Menu:** Gerencia a tela inicial do jogo.
 - `animate_entry()` - Anima a entrada do menu.

src/core/

camera.py

- **Classe Camera:** Segue o jogador durante o jogo.
 - `update()` - Ajusta a posição da câmera conforme o jogador se move.

physics.py

- **Classe Physics:** Gerencia a física dos objetos.
 - `check_collision()` - Verifica colisões entre objetos.

input.py

- **Módulo Input:** Captura entradas do teclado.
 - `get_pressed_keys()` - Retorna quais teclas estão pressionadas.

lanterna.py

- **Classe Lanterna:** Controla a visibilidade do jogador no escuro.
 - `render_light()` - Aplica um efeito de luz na tela.
-

src/data/

objects.py

- **Contém a lista de entidades do jogo, como árvores e pedras.**

sons.py

- **Armazena os efeitos sonoros utilizados no jogo.**
-

src/estagios/

estagios.py

- **Funções principais:**
 - **restart_game()** - Reinicia o jogo.
 - **game_over()** - Exibe a tela de Game Over.
 - **you_win()** – Exibe a tela de vitória
 - **main()** – Com toda a lógica de funcionamento do jogo

Nosso código tem comentários explicando as partes mais complexas de cada parte a fim de deixar mais claro nosso projeto.

Maiores desafios

- Implementar o fantasma para seguir o jogador – Resolvemos calculando a distancia entre o fantasma e o jogador e multiplicando a razão da diferença entre as distâncias pela velocidade que queríamos no fantasma

- Lógica da Lanterna – Fizemos colocando uma tela escura no mapa e calculando uma área de cone para ficar iluminada quando ligarmos a lanterna

- Cálculo para o fantasma só aparecer quando a Lanterna o iluminar – Calculamos a proximidade do fantasma com o jogador e desenhamos ele somente se a área da lanterna for igual a posição x e y do fantasma

- Colisões do jogador – Depois que entendemos o que é uma hitbox e o porquê desse nome ficou fácil, nós iniciamos ela em “Entity” com as informações de x,y,largura e altura e criamos um retângulo que colide com os demais objetos do mapa.